SchoolNet

A modern, mobile-friendly school communication MVP with Telegram-like clarity and an Instagram-like announcement feed. Built with **Python/Flask, SQLite, and vanilla HTML/CSS/JavaScript**—no Node build pipeline or paid service is needed to run it locally.

All school names, staff identities, announcements, events, and the illustrative community-member count are fictional demo data. Do not use this MVP for sensitive student information.

What works

The community feed shows staff avatars, names, exact roles, timestamps, categories, titles, and announcement content. A pinned update and a Science Club image demonstrate different content-card styles. The staff directory includes **Mr. Abebe — Physics Teacher**, **Ms. Sara — Science Club Coordinator**, and **Coach Dan — Sports Director**, each with a biography and a dedicated profile dialog.

Authorized editors can select one of these approved profiles, choose a space, and publish an announcement. The server enforces authorization and validates all fields. Published updates appear immediately and persist in SQLite across local server restarts.

Additional working interactions include category filtering, search, oldest/latest sorting, browser-scoped reaction toggles, device-local saved posts, staff profile filtering, authenticated editor comments, copy/native-share links, recent-update notifications, event details, and `.ics` calendar reminders. Calendar downloads are all-day reminders with the event time in their notes. Responsive layouts include a desktop sidebar and a mobile bottom navigation bar.

Run locally

Use **Python 3.11 or newer**. SQLite is included with standard Python. Extract the source archive and open a terminal in its `schoolnet` directory.

macOS / Linux

Bash

```
cd path/to/schoolnet
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

```
cp .env.example .env
python run.py
```

Open **http://localhost:3000**. With the blank example configuration, the app starts in reader mode.

Windows PowerShell

Plain Text

```
cd path\to\schoolnet
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
python run.py
```

If PowerShell blocks activation, you can avoid changing its execution policy by running `.\.venv\Scripts\python.exe -m pip install -r requirements.txt` and `.\.venv\Scripts\python.exe run.py` instead.

Enable local staff publishing

Stop the server. Generate two unique values using the following command twice:

Bash

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Edit `.env` and set `SCHOOLNET_PUBLISHER_KEY` to one value and `SCHOOLNET_APP_SECRET` to the other. Keep these private. Restart with `python run.py`, click **Create post**, and enter the publisher key when prompted. Select Mr. Abebe, Ms. Sara, or Coach Dan, enter a title and announcement, then click **Publish announcement**.

There is **no default publisher key**. The publisher key enables local editorial access; the application secret is recommended to keep signed local tokens valid across server restarts. If the app secret is omitted, a new random secret is generated on each process start. Local sessions last up to 12 hours and are stored in the browser tab's `sessionStorage`.

The server binds to `0.0.0.0:3000`, so on a trusted local network you can open `http://YOUR_COMPUTER_LAN_IP:3000` from a phone. A plain HTTP network connection is suitable only for this local demo, not for real credentials. To change the local port, set `PORT` in `.env`.

Architecture

Plain Text

```
schoolnet/
├── schoolnet/                  # Backend and data layer
│   ├── __init__.py             # Flask application factory and configuration
│   ├── auth.py                 # Publishing authorization, JWT/CSRF, OAuth
│   ├── db.py                   # SQLite schema, connections, queries
│   ├── routes.py               # JSON API, page, health, route manifest
│   └── seed.py                 # One-time fictional staff/posts/events
├── templates/
│   └── index.html              # Accessible application shell and dialogs
├── static/
│   ├── css/styles.css          # Responsive visual design
│   ├── js/
│   │   ├── api.js              # JSON client and preference storage
│   │   ├── app.js              # State and interaction coordination
│   │   ├── views.js            # Escaped feed/profile/calendar rendering
│   │   └── icons.js            # Inline SVG icon helpers
│   └── images/                 # Logo and local announcement photo
├── tests/test_app.py           # Isolated API/auth/persistence regressions
├── instance/                  # Created at runtime; not bundled/committed
│   └── schoolnet.db            # Local SQLite data
├── run.py                     # Development entry point
├── requirements.txt           # Pinned Python dependencies
├── Dockerfile                 # Gunicorn deployment
├── .env.example               # Empty, safe configuration template
├── ASSETS.md                  # Image/font source notes
└── TODO.md                    # Completed implementation outcomes
```

Flask serves both the frontend and JSON API from the same origin. The browser never connects directly to SQLite. Database writes use parameterized SQL; user-supplied content is escaped before HTML rendering. Staff identifiers and categories are checked on the server.

Google Fonts enhances typography when online; system-font fallbacks keep the app usable without it. JavaScript, CSS, the logo, and the feed image are served locally.

API

API successes and expected validation, authorization, missing-endpoint, and method errors use JSON. Failures use an `error` string and an appropriate HTTP status.

Method	Endpoint	Purpose

GET	`/api/bootstrap`	Profiles, feed, events, session, illustrative stats
GET	`/api/posts`	Current feed with counts and browser-specific `liked` state
POST	`/api/posts`	Publish `{profile_id, category, title, content}`
GET	`/api/posts/<id>/comments`	Read a post's comments
POST	`/api/posts/<id>/comments`	Editor comment with `{content}`
POST	`/api/posts/<id>/like`	Toggle one reaction per browser client
POST	`/api/auth/local`	Exchange `{key}` for a signed local bearer token
GET	`/api/auth/login-url?origin=...`	Get the managed Manus browser authorization URL
POST	`/api/auth/logout`	Clear the managed session cookie
GET	`/health`	Basic application/database health
GET	`/manus-routes.json`	Page route manifest

Publishing requires a trimmed title of **3–120 characters** and content of **10–3000 characters**. Categories are `Academics`, `Clubs`, `Sports`, and `General`. Approved profile IDs are `abebe`, `sara`, and `dan`. Comments require editorial permission and **1–500 characters**; the server supplies the authenticated editor's name.

Reactions are intentionally browser-scoped, not verified-person endorsements. The frontend sends a UUID `X-Client-ID`; the database's `(post_id, client_id)` primary key enforces a single toggle. GET feed/bootstrap requests with that header receive authoritative `liked` state for the same browser. Bookmarks stay on the device in `localStorage`.

Example local publishing:

Bash

```
TOKEN=$(curl -s http://localhost:3000/api/auth/local \
  -H 'Content-Type: application/json' \
  -d '{"key":"YOUR_CONFIGURED_PUBLISHER_KEY"}' \
  | python -c 'import sys,json; print(json.load(sys.stdin)["token"])')
```

```
curl -X POST http://localhost:3000/api/posts \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"profile_id":"abebe","category":"Academics","title":"Physics revision re
```

Managed Preview authentication

The hosted Manus environment uses real, validated Manus authentication rather than a simulated demo login. The app validates the `webdev_app_session` JWT or a managed bearer token with HS256, expiry, a matching `appId`, and a nonempty `openId`. Local-key login is disabled whenever managed authentication settings are present.

If Preview supplies a valid application session, publishing is available immediately; otherwise use the account menu to sign in. The normal OAuth flow uses `MANUS_OAUTH_PORTAL_URL` for browser authorization and `MANUS_OAUTH_API_URL` for token exchange and identity lookup. State is nonce-bound; failures return to the app without granting access. Public session cookies use `HttpOnly; Secure; SameSite=None`. Third-party-cookie restrictions may still prevent embedded login.

MVP permission policy: a valid signed-in Manus identity may publish under the shared approved staff profiles. Set `SCHOOLNET_PUBLISHER_OPEN_IDS` to a comma-separated allowlist to restrict editors. This is a shared editorial workflow, not individual teacher-account ownership.

Managed environment variables are `MANUS_PROJECT_ID`, `MANUS_JWT_SECRET`, `MANUS_OAUTH_PORTAL_URL`, and `MANUS_OAUTH_API_URL`. Never put their secrets in frontend code. Cookie-authenticated writes must include the identity-bound `X-CSRF-Token` from bootstrap; bearer requests do not rely on ambient cookies.

`SCHOOLNET_ALLOW_INSECURE_COOKIES=1` is only for an explicitly configured HTTP localhost OAuth flow. Keep it `0` for public HTTPS Preview/deployment.

Persistence, backup, and Docker

SQLite is seeded once with three staff profiles, four posts, and three events relative to the first initialization date. A restart does not duplicate seed content or continually change event dates. Later, older events may naturally become past events in this intentionally small demo calendar.

Data survives local server restarts on the same machine. **Container filesystems are not durable across replacement or redeployment.** No managed database was selected for this MVP. For Docker persistence, mount a volume:

Bash

```
docker build -t schoolnet .
docker run --rm -p 3000:3000 --env-file .env \
  -v schoolnet-data:/app/instance schoolnet
```

The Docker image uses Gunicorn with one worker and four threads. Back up SQLite consistently using its backup API; copying only a live `.db` file can miss WAL transactions. Stop the app before copying the full `instance/` directory, or use:

Bash

```
python -c "import sqlite3; source=sqlite3.connect('instance/schoolnet.db'); tar
```

Checks

Bash

```
python -m unittest discover -s tests -v
python -m compileall -q schoolnet run.py tests
```

The 10 regression tests use temporary databases and test-only credentials. They cover required seed identities, idempotency, input validation, unauthorized posting/comments, local access, persistent data, reaction syncing, static assets, the route manifest, health, managed JWT/CSRF, and the OAuth portal contract. They never alter the running app's database.

Before using real school data

This is a functional MVP, **not production-ready school security**. Add individual verified staff accounts with per-profile ownership, private community membership, moderation, audit logs, rate limiting, durable managed storage, monitoring, backups, and privacy/legal review before handling actual student data. The local shared key is only a lightweight editorial gate.

The app intentionally does not implement direct messaging, file uploads, student records, emergency alerts, or production push notifications. The notification button shows recent announcements within the app; it does not send messages externally. Image-source notes are in `ASSETS.md`.